BITS Pilani DIGITAL
Excellence made yours

Scikit-learn and TensorFlow for AI Engineering and MLOps

1) Big Picture: "Two Toolboxes"

Scikit-learn (sklearn)

- A Python library focused on **classical machine learning**, especially for **tabular data** (spreadsheets).
- Best known for consistent APIs:
    - fit() to train
    - predict() to infer
    - transform() to preprocess features
- Strength: **pipelines**, cross-validation, feature processing, fast baselines.

TensorFlow

- A framework for building and running **neural networks** (deep learning)
- Commonly used via **Keras** (the high-level API inside TensorFlow)
- Typical workflow:
    - define model (layers)
    - compile() (loss + optimizer + metrics)
    - fit() (training)
- Strength: deep learning, scaling, GPU support, deployment options

2) When Should Students Use Which?

Use Scikit-learn When…

- Your data is **rows/columns** (tabular): churn, credit risk, demand forecasting
- You want strong baselines quickly
- You need solid preprocessing + evaluation tools in one place

Use TensorFlow When…

- Your data is **unstructured**: images, audio, text at scale
- You need **neural networks** or transfer learning
- You want GPU acceleration or custom model architectures

Common Real-World Pattern:

- scikit-learn for: preprocessing, metrics, quick baselines
- TensorFlow for: neural network training/inference

3) Scikit-learn Essentials

A) The "fit / predict" Pattern

- **Train**: model.fit(X_train, y_train)
- **Predict**: model.predict(X_test)

B) Pipelines

A Pipeline chains preprocessing + model so you don't forget steps at inference time.

C) Model Selection

- Train/test split

- Cross-validation
- Hyperparameter search (Grid/Random)

4) TensorFlow Essentials

A) Model = Layers + Weights

Students should see neural nets as “a function with learnable parameters”.

B) Compile Step

- **Loss**: How wrong the model is (objective)
- **Optimizer**: How weights update (learning rule)
- **Metrics**: What you monitor (accuracy, etc.)

C) Training Loop

Keras hides the training loop; you typically use model.fit().

5) Side-by-side simple example (same dataset)

Example Task: Classify Iris Flowers

A) Scikit-learn Baseline (Logistic Regression + Pipeline)

Python

```python
# pip install scikit-learn
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score
X, y = load_iris(return_X_y=True)
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)
model = make_pipeline(
    StandardScaler(),
    LogisticRegression(max_iter=1000)
)
model.fit(X_train, y_train)
pred = model.predict(X_test)
print("Accuracy:", accuracy_score(y_test, pred))
```

- Pipeline = “preprocess + model in one object”
- This is the fastest way to get a strong baseline on tabular problems

B) TensorFlow Neural Net (Keras)

Python

```python
# pip install tensorflow scikit-learn
import tensorflow as tf
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
X, y = load_iris(return_X_y=True)
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
model = tf.keras.Sequential([
    tf.keras.layers.Dense(16, activation="relu", input_shape=(X_train.shape[1],)),
    tf.keras.layers.Dense(3, activation="softmax")
])
model.compile(
    optimizer="adam",
    loss="sparse_categorical_crossentropy",
    metrics=["accuracy"]
)
model.fit(X_train, y_train, epochs=30, verbose=0)
loss, acc = model.evaluate(X_test, y_test, verbose=0)
print("Accuracy:", acc)
```

- TensorFlow adds extra choices: architecture, epochs, optimizer
- More flexibility; more tuning responsibility

6) “Famous” Algorithm Categories in Scikit-learn

Supervised: Classification

- Logistic Regression
- SVM
- Decision Trees / Random Forest
- Gradient Boosting
- Naive Bayes
- kNN

Supervised: Regression

- Linear / Ridge / Lasso / ElasticNet
- Random Forest Regressor
- Gradient Boosting Regressor
- SVR

Unsupervised

- Clustering: k-means, DBSCAN, Agglomerative
- Dimensionality reduction: PCA, TruncatedSVD

- Anomaly detection: Isolation Forest, One-Class SVM

7) Cheat Sheet: Minimal Mental Model

- **sklearn**: Pipeline(preprocess, model).fit(X,y) → stable baseline, easy MLOps
- **TensorFlow**: define → compile → fit → neural nets, scale, more knobs

Topic	TensorFlow	Scikit-learn
Main focus	Deep learning + end-to-end ML platform (TensorFlow)	"Classic" machine learning algorithms + preprocessing + evaluation tools (Scikit-learn)
Best for	Neural nets: vision, NLP, audio, large models	Tabular data: regression/classification, clustering, feature engineering
Typical models	CNNs, RNNs, Transformers, custom neural nets	Logistic regression, SVMs, random forests, gradient boosting, k-means, etc. (Wikipedia)
Hardware acceleration	Strong GPU/TPU support (common in DL workflows)	Mostly CPU-focused (some algorithms can benefit from optimised math libs)
API feel	More "build a computation + train loop"; often via **Keras** high-level API (TensorFlow)	Very consistent "fit / predict / transform" style across models and pipelines (Scikit-learn)
Deployment	Multiple deployment paths (server/mobile/edge), designed for production workflows	Great for model training & evaluation; deployment typically via Python + pickle/joblib and an app stack